

Univerzitet u Sarajevu
Elektrotehnički fakultet u Sarajevu
Odsjek za računarstvo i informatiku



Drugi dio projektnog zadatka
Verifikacija i validacija softvera

Amina Čajić, 19218

Član tima: Amina Čajić

Kod odabrane metode:

```
public string ObrisiNeprimjereneRecenzije(int idKnjige, string kljucneRijeci)
        {
            var k = _db.GetKnjiga(idKnjige);
            if (k == null) { return "GREŠKA: Knjiga nije pronađena."; }
            if (k.Recenzije == null || k.Recenzije.Count == 0) { return "Nema recenzija
za ovu knjigu."; }

            if (IsNullOrWhite(kljucneRijeci))
            {
                return "Ključne riječi su prazne ili sadrže samo razmake. Nema
brisanja.";
            }

            var tokens = kljucneRijeci.Split(',',
StringSplitOptions.RemoveEmptyEntries);

            var recenzijeZaBrisanje = new List<Recenzija>();

            for (int i = 0; i < k.Recenzije.Count; i++)
            {
                var recenzija = k.Recenzije[i];
                var komentar = recenzija.Komentar ?? "";

                for (int t = 0; t < tokens.Length; t++)
                {
                    var token = tokens[t].Trim();
                    if (CaseInsensitiveContains(komentar, token))
                    {
                        recenzijeZaBrisanje.Add(recenzija);
                        break;
                    }
                }
            }

            int removed = recenzijeZaBrisanje.Count;

            foreach (var recenzija in recenzijeZaBrisanje)
            {
                k.Recenzije.Remove(recenzija);

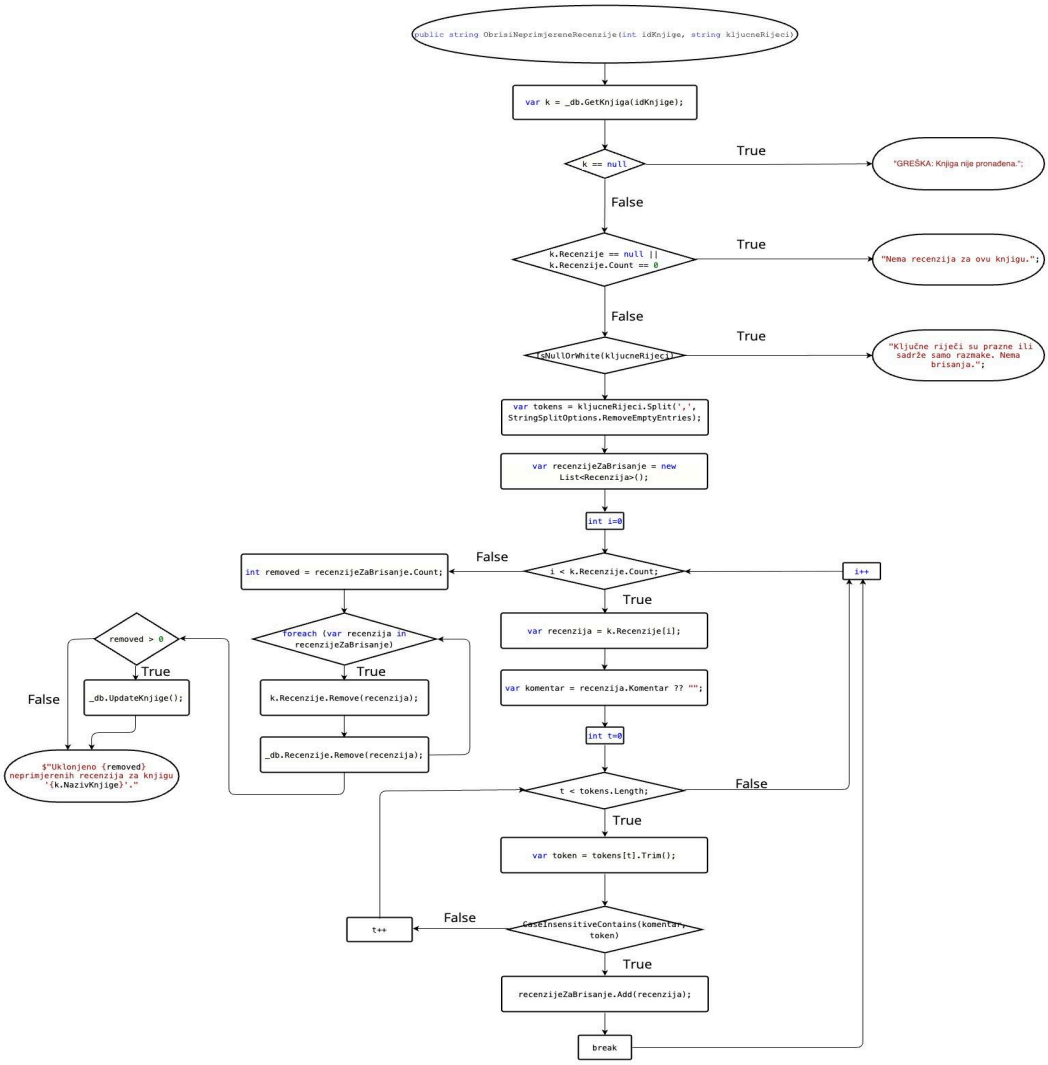
                _db.Recenzije.Remove(recenzija);
            }

            if(removed > 0)
                _db.UpdateKnjige();

            return $"Uklonjeno {removed} neprimjerenih recenzija za knjigu
'{k.NazivKnjige}'.";
        }
```

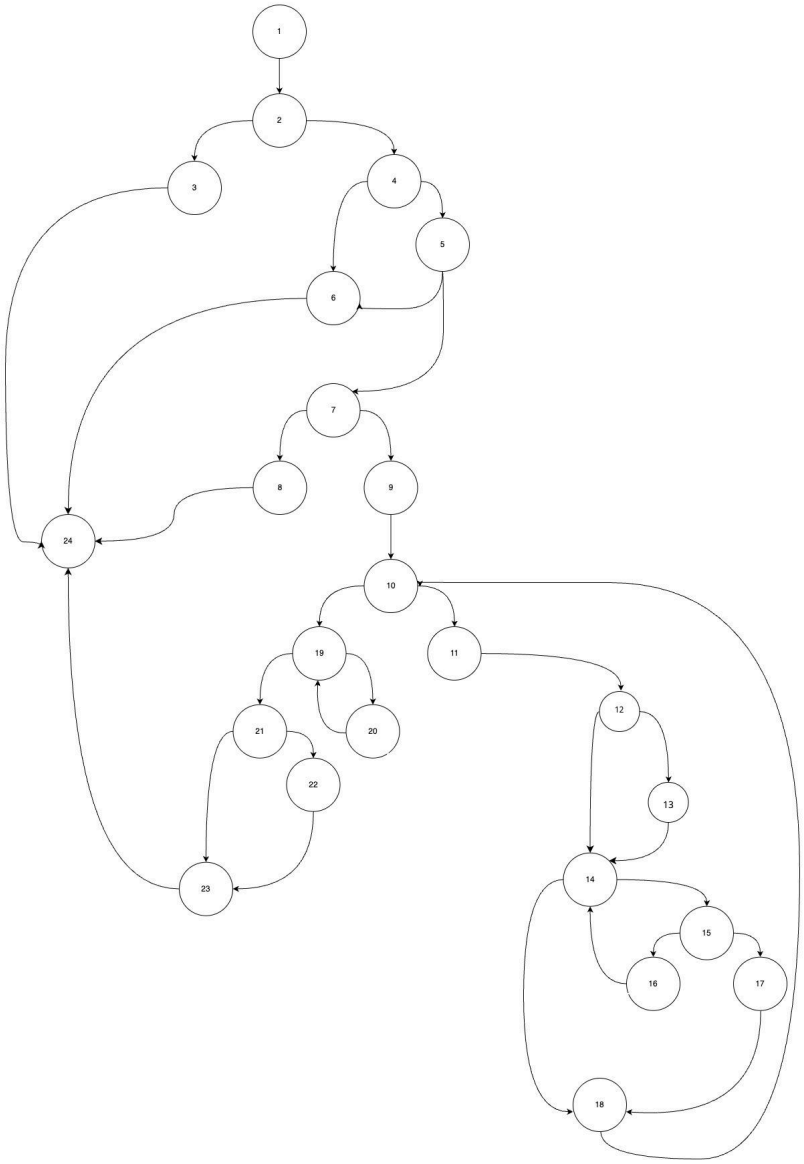
1. Izračunati McCabe metriku i nacrtati kontrolni graf. Uporediti rezultate sa rezultatima iz Code Metrics alata i prodiskutovati kvalitet koda kroz metrike.

Izgled kontrolnog grafa za metodu:



Slika 1 - Kontrolni graf

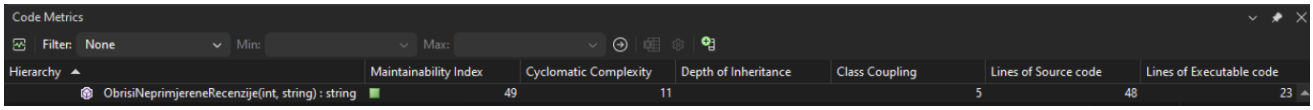
Made with VisualParadigm
For non-commercial use



Slika 2 – Flow graf

e = 33 je broj tokova u grafu programskog toka tj. broj grana
n = 24 je broj čvorova u grafu programskog toka
p = 1 je broj formiranih neovisnih dijelova grafova
Generalna formula za računanje McCabe metrike kompleksnosti:

$$M = V(G) = e - n + 2p = 33 - 24 + 2*1 = 11$$



Slika 3 - Code Metrics u Visual Studio-u

Na slici 3 se vidi da su korištenjem alata Code Metrics u Visual Studio-u dobiveni sljedeći podaci:

- Cyclomatic Complexity: 11
- Maintainability Index: 49
- Class Coupling: 5
- Lines of Source Code: 48

McCabe metrika primjenjena direktno na programski kod zasniva se na računanju tačaka odluke u modulu.

```
public string ObrisiNeprimjereneRecenzije(int idKnjige, string kljucneRijeci){

            var k = _db.GetKnjiga(idKnjige);
            if(1) (k == null) { return "GREŠKA: Knjiga nije pronađena."; }
            if(2) (k.Recenzije == null ||(3) k.Recenzije.Count == 0) { return "Nema recenzija za
ovu knjigu."; }

            if(4) (IsNullOrWhite(kljucneRijeci))
            {
                return "Ključne riječi su prazne ili sadrže samo razmake. Nema brisanja.";
            }

            var tokens = kljucneRijeci.Split(',', StringSplitOptions.RemoveEmptyEntries);

            var recenzijeZaBrisanje = new List<Recenzija>();

            for(5) (int i = 0; i < k.Recenzije.Count; i++)
            {
                var recenzija = k.Recenzije[i];
                var komentar = recenzija.Komentar ??(6) "";

                for(7) (int t = 0; t < tokens.Length; t++)
                {
                    var token = tokens[t].Trim();
                    if(8) (CaseInsensitiveContains(komentar, token))
                    {
                        recenzijeZaBrisanje.Add(recenzija);
                        break;
                    }
                }
            }
            int removed = recenzijeZaBrisanje.Count;

            foreach(9) (var recenzija in recenzijeZaBrisanje)
            {
                k.Recenzije.Remove(recenzija);

                _db.Recenzije.Remove(recenzija);
            }

            if(10)(removed > 0)
                _db.UpdateKnjige();

            return $"Uklonjeno {removed} neprimjerenih recenzija za knjigu '{k.NazivKnjige}'.";
        }
```

Tehnika je zasnovana na 3 koraka:

- Korak 1: Počinje se sa 1.
  -> 1
- Korak 2: Dodati 1 za svaku od sljedećih ključnih riječi ili njihovih ekvivalenata: if, while, repeat, for, and, or.
  -> + 10
- Korak 3: Dodati 1 za svaki case u case iskazu.
  -> + 0
  -> = 11

Vidljivo je da se rezultat dobiven automatskom statičkom analizom sa Code Metrics unutar Visual Studio-a podudara sa rezultatom dobijenim manuelnom tehnikom brojanja tačaka odluke direktno iz koda, kao i primjenom formule za računanje McCabe metrike kompleksnosti M = V(G) = e - n + 2p

Na slici 3 se vidi da je alat Code Metrics je za ovu metodu izračunao indeks održavanja (Maintainability Index) od 49. Prema korištenoj skali:

Index value	Color	Meaning
0-9	Red	Low maintainability of code
10-19	Yellow	Moderate maintainability of code
20-100	Green	Good maintainability of code

ova metoda spada u kategoriju "Dobra održivost".

Kako veći indeks održavanja podrazumijeva da je metoda lakša za održavanje, nastojat ćemo da code tuning-om i refaktoringom postignemo povećanje ovog indeksa.

2. Uraditi white box testiranje nad metodom koristeći sve pristupe white box[1] testiranja koja su primjenjiva na metodu. Prikazati odabrane metode, dizajnirane testne slučajeve, i provedeni postupak testiranja.

Made with VisualParadigm For non-commercial use

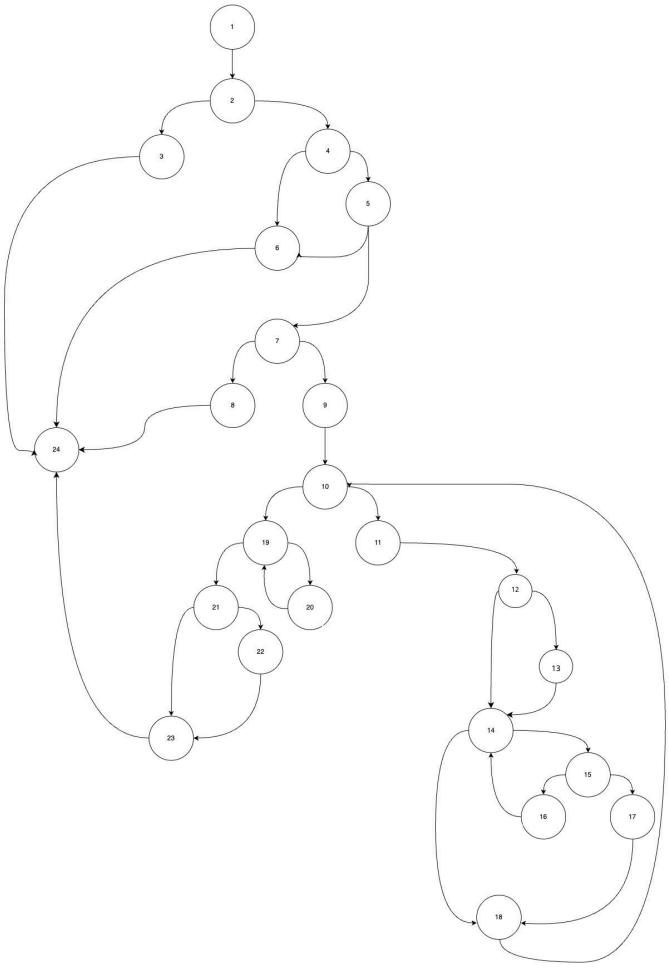


Tabela na osnovu grafa:

Redni broj puta	Čvorovi koje put obuhvata
1	1-2-3-24
2	1-2-4-6-24
3	1-2-4-5-6-24
4	1-2-4-5-7-8-24
5	1-2-4-5-7-9-10-19-21-23-24

6	1-2-4-5-7-9-10-19-21-22-23-24
7	1-2-4-5-7-9-10-19-20-19-21-23-24
8	1-2-4-5-7-9-10-19-20-19-21-22-23-24
9	1-2-4-5-7-9-10-11-12-14-18-10-19-21-23-24
10	1-2-4-5-7-9-10-11-12-14-18-10-19-21-22-23-24
11	1-2-4-5-7-9-10-11-12-14-18-10-19-20-19-21-23-24
12	1-2-4-5-7-9-10-11-12-14-18-10-19-20-19-21-22-23-24
13	1-2-4-5-7-9-10-11-12-13-14-18-10-19-21-23-24
14	1-2-4-5-7-9-10-11-12-13-14-18-10-19-21-22-23-24
15	1-2-4-5-7-9-10-11-12-13-14-18-10-19-20-19-21-23-24
16	1-2-4-5-7-9-10-11-12-13-14-18-10-19-20-19-21-22-23-24
17	1-2-4-5-7-9-10-11-12-**14-15-16-14**-18-10-19-21-23-24
18	1-2-4-5-7-9-10-11-12-**14-15-16-14**-18-10-19-21-22-23-24
19	1-2-4-5-7-9-10-11-12-**14-15-16-14**-18-10-19-20-19-21-23-24
20	1-2-4-5-7-9-10-11-12-**14-15-16-14**-18-10-19-20-19-21-22-23-24
21	1-2-4-5-7-9-10-11-12-13-**14-15-16-14**-18-10-19-21-23-24
22	1-2-4-5-7-9-10-11-12-13-**14-15-16-14**-18-10-19-21-22-23-24
23	1-2-4-5-7-9-10-11-12-13-**14-15-16-14**-18-10-19-20-19-21-23-24
24	1-2-4-5-7-9-10-11-12-13-**14-15-16-14**-18-10-19-20-19-21-22-23-24
25	1-2-4-5-7-9-10-11-12-**14-15-16-14**-15-17-18-10-19-21-23-24
26	1-2-4-5-7-9-10-11-12-**14-15-16-14**-15-17-18-10-19-21-22-23-24
27	1-2-4-5-7-9-10-11-12-**14-15-16-14**-15-17-18-10-19-20-19-21-23-24
28	**1**-2-4-5-7-9-10-11-12-**14-15-16-14**-15-17-18-10-19-20-19-21-22-23-24
29	1-2-4-5-7-9-10-11-12-13-**14-15-16-14**-15-17-18-10-19-21-23-24
30	1-2-4-5-7-9-10-11-12-13-**14-15-16-14**-15-17-18-10-19-21-22-23-

	24
31	1-2-4-5-7-9-10-11-12-13-**14-15-16-14**-15-17-18-10-19-20-19-21-23-24
32	1-2-4-5-7-9-10-11-12-13-**14-15-16-14-**15-17-18-10-19-20-19-21-22-23-24
33	1-2-4-5-7-9-10-11-12-14-15-17-18-10-19-21-23-24
34	1-2-4-5-7-9-10-11-12-14-15-17-18-10-19-21-22-23-24
35	1-2-4-5-7-9-10-11-12-14-15-17-18-10-19-20-19-21-23-24
36	1-2-4-5-7-9-10-11-12-14-15-17-18-10-19-20-19-21-22-23-24
37	1-2-4-5-7-9-10-11-12-13-14-15-17-18-10-19-21-23-24
38	1-2-4-5-7-9-10-11-12-13-14-15-17-18-10-19-21-22-23-24
39	1-2-4-5-7-9-10-11-12-13-14-15-17-18-10-19-20-19-21-23-24
40	1-2-4-5-7-9-10-11-12-13-14-15-17-18-10-19-20-19-21-22-23-24

White-box testiranje – pristupi planiranju testnih slučajeva:

1. Obuhvat linija
2. Obuhvat grana / odluka
3. Obuhvat uslova
4. Modifikovani uslov / odluka obuhvat
5. Obuhvat petlji
6. Obuhvat toka podataka
7. Obuhvat puteva

Obuhvat linija

Obuhvat linija ima težnju da dizajnira testne slučajeve tako da se svaki iskaz, tj. svaka linija koda, izvrši barem jednom. Pokrivenost iskaza = ukupan broj iskaza obuhvaćen testovima u odnosu na ukupan broj iskaza u modulu. Cilj je postići 100% pokrivenost iskaza uz minimalan broj testnih slučajeva-testova.
Kako bi se ovo postiglo, potrebna su 4 testa:

- Test 1: Traži se knjiga koja ne postoji u bazi
  - Put: 1 - 2 - 3 - 24

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_KnjigaNePostoji_VracaGresku()
{
    string rezultat = _servis.ObrisiNeprimjereneRecenzije(999, "uvredljivo");
    StringAssert.Contains(rezultat, "GREŠKA: Knjiga nije pronađena.");
}
```

- Test 2: Knjiga postoji, ali lista njenih recenzija je inicijalizovana i prazna (Count == 0).

- Put: 1 - 2 - 4 - 5 - 6 - 24

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_PraznaListaRecenzija_VracaPoruku()
{
    var k = new Knjiga(1, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    k.Recenzije = new List<Recenzija>();
    _db.knjige.Add(k);

    string rezultat = _servis.ObrisiNeprimjereneRecenzije(1, "uvreda");

    StringAssert.Contains(rezultat, "Nema recenzija za ovu knjigu.");
    Assert.AreEqual(0, k.Recenzije.Count);
}
```

- Test 3: Proslijeđen prazan string ili razmak za ključne riječi (" ")
  - Put: 1 - 2 - 4 - 5 - 7 - 8 - 24

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_PrazneKljucneRijeci_VracaPoruku()
{
    var k = new Knjiga(1, "12345", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    k.Recenzije = new List<Recenzija> { new Recenzija { ID = 1, Komentar = "Neki komentar" } };
    _db.knjige.Add(k);

    string rezultat = _servis.ObrisiNeprimjereneRecenzije(1, " ");
    StringAssert.Contains(rezultat, "Ključne riječi su prazne ili sadrže samo razmake. Nema brisanja.");
    Assert.AreEqual(1, k.Recenzije.Count);
}
```

- Test 4: Postoje recenzije koje sadrže ključne riječi; vrši se brisanje 2 recenzije
  - Put: 1 - 2 - 4 - 5 - 7 - 9 - 10 - 11 - 12 - 13 - 14 - 15 - 16 - 14 - 15 - 17 - 18 - 10 - 19 - 20 - 19 - 21 - 22 - 23 - 24

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_UklanjaRecenzije()
{
    var k = new Knjiga(1, "12345", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    var recenzijaZaBrisanje1 = new Recenzija { ID = 1, KnjigaID = 1, Komentar = "Uvredljivo sadrzaj" };
    var recenzijaZaBrisanje2 = new Recenzija { ID = 2, KnjigaID = 1, Komentar = "Drugi Uvredljiv komentar" };
    var recenzijaOK = new Recenzija { ID = 3, KnjigaID = 1, Komentar = "Normalna recenzija" };

    k.Recenzije = new List<Recenzija> { recenzijaZaBrisanje1, recenzijaZaBrisanje2, recenzijaOK };
    _db.Recenzije.AddRange(k.Recenzije);
    _db.knjige.Add(k);

    string rezultat = _servis.ObrisiNeprimjereneRecenzije(1, "uvredljivo,komentar");

    Assert.AreEqual(1, k.Recenzije.Count);
    Assert.AreEqual(1, _db.Recenzije.Count);
    Assert.AreEqual("Normalna recenzija", k.Recenzije[0].Komentar);
    StringAssert.Contains(rezultat, "Uklonjeno 2 neprimjerenih recenzija za knjigu 'Naziv'.");
}
```

Obuhvat grana / odluka

Obuhvat grana/odluka - dizajnira se minimalan broj testnih slučajeva koji imaju za cilj da se svi mogući ishodi odluka (grane) u kodu izvrše. Ova strategija je strožija u odnosu na strategiju obuhvata iskaza. Odluke u kodu uključuju: if, while, do while, for, switch… Npr. if iskaz / while se testira i sa pozitivno i sa negativno procijenjenim povezanim logičkim uslovom.
Switch iskaz se testira tako da se testira svaka case i default grana. Ako ne postoji default grana switch se testira i sa vrijednošću koja ne odgovara nijednoj case grani.

Za potpunu pokrivenost svih grana dovoljno je definisati 6 specifičnih testnih puteva.

Redni broj puta	Niz čvorova	Da li se pravi test
1	1 - 2 - 3 - 24	NE
2	1 - 2 - 4 - 6 - 24	DA
3	1 - 2 - 4 - 5 - 6 - 24	NE
4	1 - 2 - 4 - 5 - 7 - 8 - 24	NE
5	1-2-4-5-7-9-10-11-12-13-14-15-17-18-10-19-20-19-21-22-23-24	NE (**(ObrisiNeprimjereneRecenzije_UklanjaRecenzije**)
6	1-2-4-5-7-9-10-11-12-14-15-16-14-18-10-19-21-23-24	DA KomentarNull

Odluke	Ishod: T	Ishod: F
Čvor 2 (k == null)	Put 1	Put 2
Čvor 4 (k.Recenzije == null)	Put 2	Put 3
Čvor 5 (k.Recenzije.Count == 0)	Put 3	Put 4
Čvor 7 (IsNullOrWhite)	Put 4	Put 5
Čvor 10 (i < k.Recenzije.Count)	Put 5 , 6	Put 5, 6
Čvor 12 (recenzija.Komentar != null)	Put 5	Put 6
Čvor 14 (t < tokens.Length)	Put 5, 6	Put 6
Čvor 15 (Contains)	Put 5	Put 6
Čvor 19 (foreach recenzija)	Put 5	Put 5, 6
Čvor 21 (removed > 0)	Put 5	Put 6

Za 100% obuhvat grana neophodni su sljedeći dodatni testovi:

- Test 1: Lista k.Recenzije je eksplicitno null
  - Put: 1-2-4-6-24

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_ListaRecenzijaNull_VracaPoruku()
{
    var k = new Knjiga(2, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    k.Recenzije = null;
    _db.knjige.Add(k);

    string rezultat = _servis.ObrisiNeprimjereneRecenzije(2, "uvreda");

    StringAssert.Contains(rezultat, "Nema recenzija za ovu knjigu.");
}
```

- Test 2: Postoji recenzija, ali joj je polje Komentar jednako null
    - Put: 1-2-4-5-7-9-10-11-12-14-15-16-14-18-10-19-21-23-24

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_KomentarNull()
{
    var k = new Knjiga(7, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    k.Recenzije = new List<Recenzija> { new Recenzija { ID = 10, Komentar = null } };
    _db.knjige.Add(k);

    string rezultat = _servis.ObrisiNeprimjereneRecenzije(7, "uvreda");

    Assert.AreEqual(1, k.Recenzije.Count);
    StringAssert.Contains(rezultat, "Uklonjeno 0");
}
```

- Test 3: Postoje recenzije, ali nijedna ne sadrži zabranjene riječi.
    - Put: 1-2-4-5-7-9-10-11-12-13-14-15-16-14-18-10-19-21-23-24

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_NemaPodudaranja()
{
    var k = new Knjiga(8, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    k.Recenzije = new List<Recenzija> { new Recenzija { ID = 11, Komentar = "Ovo je sasvim pristojan komentar." } };
    _db.knjige.Add(k);

    string rezultat = _servis.ObrisiNeprimjereneRecenzije(8, "uvreda,spam");

    Assert.AreEqual(1, k.Recenzije.Count);
    StringAssert.Contains(rezultat, "Uklonjeno 0");
}
```

Obuhvat uslova

Obuhvat uslova je tehnika testiranja koja procjenjuje svaki pojedinačni logički uslov unutar složenih izraza u iskazima selekcije i petljama (npr. if, while, do...while).
Iako obuhvat grana osigurava da su sve grane svake odluke izvršene barem jednom, on ne garantuje da je svaki elementarni uslov unutar složenih logičkih izraza evaluiran i kao true i kao false. Obuhvat uslova zahtijeva da svaki pojedinačni logički uslov dobije obje vrijednosti, čak i kada više uslova vodi ka istom ishodu odluke. Za razliku od obuhvata grana (koji posmatra cijeli uslov kao jednu cjelinu, te uzima u obzir samo konačan ishod) obuhvat uslova gleda svaki pojedinačni pod-uslov unutar zagrada.

U obuhvatu grana su napisani testovi koji osiguravaju da se svaka odluka u programu izvrši barem jednom sa pozitivnim i negativnim ishodom:

Odluka	Ishod: T	Ishod: F
Čvor 2 (k == null)	KnjigaNePostoji_VracaGresku	UklanjaRecenzije
Čvor 4 (k.Recenzije == null)	ListaRecenzijaNull_VracaPoruku	UklanjaRecenzije
Čvor 5 (k.Recenzije.Count == 0)	PraznaListaRecenzija_VracaPoruku	UklanjaRecenzije
Čvor 7 (IsNullOrWhite)	PrazneKljucneRijeci_VracaPoruku	UklanjaRecenzije
Čvor 10 (i < k.Recenzije.Count)	UklanjaRecenzije	UklanjaRecenzije (nakon zadnje iteracije)
Čvor 12 (recenzija.Komentar != null)	UklanjaRecenzije	KomentarNull
Čvor 14 (t < tokens.Length)	UklanjaRecenzije	UklanjaRecenzije (nakon zadnje iteracije
Čvor 15 (Contains)	UklanjaRecenzije	NemaPodudaranja
Čvor 19 (foreach recenzija)	UklanjaRecenzije	UklanjaRecenzijen (nakon zadnje iteracije)
Čvor 21 (removed > 0)	UklanjaRecenzije	NemaPodudaranja

Međutim, kako bi se postigao obuhvat uslova, potrebno je dodatno analizirati pojedinačne elementarne uslove složenog uslova if (k.Recenzije == null || k.Recenzije.Count == 0). Za razmatranje svih mogućih kombinacija vrijednosti elementarnih uslova i njihovog uticaja na konačan ishod izraza koristila sam truth tabelu:

k.Recenzije == null	k.Recenzije.Count == 0	Ishod cijelog izraza
T	Ne evaluira se	T
F	T	T
F	F	F

Na osnovu prikazane truth tabele se može zaključiti da su za potpuni obuhvat ovog uslova neophodna tri testna slučaja (dok su u slučaju obuhvata grana bila potrebna dva):

Testni slučaj	Put	Pokriven testom
Lista recenzija je NULL (k.Recenzije == null (T), k.Recenzije.Count == 0 se ne evaluira)	1 - 2 - 4(T) - 6 - 24	ObrisiNeprimjereneRecenzije_ListaRecenzijaNull_VracaPoruku
Lista recenzija je inicijalizirana, ali nema elemenata (k.Recenzije == null (F), k.Recenzije.Count == 0 (T))	1 - 2 - 4(F) - 5(T) - 6 - 24	ObrisiNeprimjereneRecenzije_PraznaListaRecenzija_VracaPoruku
Lista recenzija postoji i nije prazna (k.Recenzije == null (F), k.Recenzije.Count == 0 (F))	1 - 2 - 4(F) - 5(F) - 7 - …	ObrisiNeprimjereneRecenzije_UklanjaRecenzije i ObrisiNeprimjereneRecenzije_NemaPodudaranja

Modifikovani uslov/odluka obuhvat (MC/DC)

MC/DC kombinuje prednosti obuhvata odluka i obuhvata uslova, ali smanjuje broj testova. Zahtijeva da svaki elementarni uslov unutar složene odluke može samostalno uticati na promjenu ishoda odluke, dok se ostali uslovi drže konstantnim. Svaki uslov mora barem jednom promijeniti odluku sa True na False ili obrnuto.
MC/DC je relevantan samo za odluke koje sadrže više od jednog elementarnog uslova, pa ćemo MC/DC analizu primijeniti na: if (k.Recenzije == null || k.Recenzije.Count == 0) .

Cilj MC/DC je dokazati da elementarni uslovi k.Recenzije == null i k.Recenzije.Count == 0 mogu samostalno promijeniti ishod odluke, pri čemu drugi uslov ostaje konstantan.

Samostalan uticaj	Ishod	Pokriven testom
uslova k.Recenzije == null	T	ObrisiNeprimjereneRecenzije_ListaRecenzijaNull_VracaPoruku
k.Recenzije.Count == 0	T	ObrisiNeprimjereneRecenzije_PraznaListaRecenzija_VracaPoruku
k.Recenzije == null (F), k.Recenzije.Count == 0 (F)	F	ObrisiNeprimjereneRecenzije_UklanjaRecenzije i ObrisiNeprimjereneRecenzije_NemaPodudaranja

Za ostale odluke u metodi MC/DC se trivijalno svodi na obuhvat odluke, jer sadrže samo jedan elementarni uslov.

Dakle, MC/DC je postignut bez dodavanja novih testova, koristeći već postojeće testne slučajeve definisane u prethodnim fazama testiranja.

Obuhvat petlji

Jednostavna petlja sa n prolaza
- preskočiti unutrašnjost petlje (0 prolaza)
- jedan prolazak kroz petlju
- dva prolaska kroz petlju
- m prolazaka kroz petlju gdje je m<n (slučajan broj prolaza ispod maksimalnog broja)
- (n-1), (n), (n+1) prolazaka kroz petlju (granični slučajevi)

Ugnježdene petlje
- početak testiranja sa unutrašnjom petljom; sve ostale petlje se postavljaju na minimalni broj prolaz
- primijeniti iste testove kao za jednostavnu petlju sa n prolaza
- ponavljati isti postupak za sve vanjske petlje

Povezane petlje
- ako su petlje neovisne, testira se svaka zasebno kao jednostavna petlja sa n prolaza
- ako su petlje ovisne tretiraju se kao ugniježdene petlje

Petlje u metodi ObrisiNeprimjereneRecenzije:

1. Vanjska for petlja: for (int i = 0; i < k.Recenzije.Count; i++)
2. Unutrašnja for petlja: for (int t = 0; t < tokens.Length; t++)
3. Foreach petlja: foreach (var recenzija in recenzijeZaBrisanje

Broj prolazaka unutrašnje petlje zavisi od broja ključnih riječi (tokens.Length).

Broj prolazaka	Opis
0	Nema ključnih riječi
1	Jedna ključna riječ
2	Dvije ključne riječi
m < n	5 ključnih riječi, a pronađe se podudaranje u n-tom prolazu kroz petlju
n	Podudaranje se nađe sa zadnjom ključnom riječi, u 5-tom prolazu kroz petlju Ili se podudaranje ne pronađe
n+1	Nije moguće

Broj prolazaka vanjske petlje zavisi od broja recenzija (k.Recenzije.Count):

Broj prolazaka	Opis
0	Lista recenzija prazna
1	Jedna recenzija
2	Dvije recenzije
n	5 recenzija

Testirana je svaka kombinacija broja prolazaka vanjske petlje sa svakim brojem prolazaka unutrašnje petlje:
Već pokrivene kombinacije:

1. Nema ključnih riječi i jedna recenzija (tokens.Length=0, k.Recenzije.Count=1 u ObrisiNeprimjereneRecenzije_PrazneKljucneRijeci_VracaPoruku
2. Jedna ključna riječ i lista recenzija prazna (tokens.Length=1, k.Recenzije.Count=0) u ObrisiNeprimjereneRecenzije_PraznaListaRecenzija_VracaPoruku
3. Dvije ključne riječi i jedna recenzija (tokens.Length=2, k.Recenzije.Count=1) u ObrisiNeprimjereneRecenzije_NemaPodudaranja

Testovi za ostale kombinacije:

- Test 1: Nema ključnih riječi i lista recenzija prazna (tokens.Length=0,k.Recenzije.Count=0)

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_PrazniTokeniIPrazneRecenzije()
{
    var k = new Knjiga(10, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    k.Recenzije = new List<Recenzija>();
    _db.knjige.Add(k);
    string rezultat = _servis.ObrisiNeprimjereneRecenzije(10, " ");
    StringAssert.Contains(rezultat, "Nema recenzija za ovu knjigu.");
    Assert.AreEqual(0, k.Recenzije.Count);
}
```

- Test 2: Nema ključnih riječi i dvije recenzije (tokens.Length=0, k.Recenzije.Count=2)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PrazniTokeniSaDvijeRecenzije()
    {
        var k = new Knjiga(11, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 20, Komentar = "Recenzija 1" },
    new Recenzija { ID = 21, Komentar = "Recenzija 2" }
};
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(11, "   ");
        StringAssert.Contains(rezultat, "Ključne riječi su prazne ili sadrže samo razmake");
        Assert.AreEqual(2, k.Recenzije.Count);
    }
```

- Test 3: Nema ključnih riječi i 5 recenzija
  (tokens.length=0, k.Recenzije.Count=5)

```
    // tokens.length=0, k.Recenzije.Count=5
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PrazniTokeniSaPetRecenzija()
    {
        var k = new Knjiga(12, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 30, Komentar = "Rec1" },
    new Recenzija { ID = 31, Komentar = "Rec2" },
    new Recenzija { ID = 32, Komentar = "Rec3" },
    new Recenzija { ID = 33, Komentar = "Rec4" },
    new Recenzija { ID = 34, Komentar = "Rec5" }
};
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(12, "");
        StringAssert.Contains(rezultat, "Ključne riječi su prazne ili sadrže samo razmake");
        Assert.AreEqual(5, k.Recenzije.Count);
    }
```

- Test 4: Jedna ključna riječ i jedna recenzija
  (tokens.length=1, k.Recenzije.Count=1)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_JedanTokenJednaRecenzijaSaPodudaranjem()
    {
        var k = new Knjiga(13, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 40, KnjigaID = 13, Komentar = "Spam poruka" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(13, "spam");
        Assert.AreEqual(0, k.Recenzije.Count);
        Assert.AreEqual(0, _db.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 1");
    }
```

- Test 5: Jedna ključna riječ i dvije recenzije
  (tokens.length=1, k.Recenzije.Count=2)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_JedanTokenDvijeRecenzije()
    {
        var k = new Knjiga(14, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 50, KnjigaID = 14, Komentar = "Loše spam" },
    new Recenzija { ID = 51, KnjigaID = 14, Komentar = "Dobra recenzija" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(14, "spam");
        Assert.AreEqual(1, k.Recenzije.Count);
        Assert.AreEqual(1, _db.Recenzije.Count);
        Assert.AreEqual("Dobra recenzija", k.Recenzije[0].Komentar);
        StringAssert.Contains(rezultat, "Uklonjeno 1");
    }
```

- Test 6: Jedna ključna riječ i 5 recenzija
  (tokens.length=1, k.Recenzije.Count=5)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_JedanTokenPetRecenzija()
    {
        var k = new Knjiga(15, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 60, KnjigaID = 15, Komentar = "Užasno loše" },
    new Recenzija { ID = 61, KnjigaID = 15, Komentar = "Odlično" },
    new Recenzija { ID = 62, KnjigaID = 15, Komentar = "Užasno loša knjiga" },
    new Recenzija { ID = 63, KnjigaID = 15, Komentar = "Super" },
    new Recenzija { ID = 64, KnjigaID = 15, Komentar = "Preporučujem" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(15, "užasno");
        Assert.AreEqual(3, k.Recenzije.Count);
        Assert.AreEqual(3, _db.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 2");
    }
```

- Test 7: Dvije ključne riječi i lista recenzija prazna
  (tokens.length=2, k.Recenzije.Count=0)

```
[TestMethod]
0 references
public void ObrisiNeprimjereneRecenzije_DvaTokenaPrazneRecenzije()
{
    var k = new Knjiga(16, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    k.Recenzije = new List<Recenzija>();
    _db.knjige.Add(k);
    string rezultat = _servis.ObrisiNeprimjereneRecenzije(16, "spam,uvreda");
    StringAssert.Contains(rezultat, "Nema recenzija za ovu knjigu");
    Assert.AreEqual(0, k.Recenzije.Count);
}
```

- Test 8: Dvije ključne riječi i dvije recenzije
  (tokens.length=2, k.Recenzije.Count=2)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_DvaTokenaDvijeRecenzijeSaPodudaranjem()
    {
        var k = new Knjiga(17, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 70, KnjigaID = 17, Komentar = "Spam poruka" },
    new Recenzija { ID = 71, KnjigaID = 17, Komentar = "Uvredljiv sadržaj" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(17, "spam,uvredljiv");
        Assert.AreEqual(0, k.Recenzije.Count);
        Assert.AreEqual(0, _db.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 2");
    }
```

- Test 9: Dvije ključne riječi i 5 recenzija
  (tokens.length=2, k.Recenzije.Count=5)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_DvaTokenaPetRecenzija()
    {
        var k = new Knjiga(18, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 80, KnjigaID = 18, Komentar = "Spam ovdje" },
    new Recenzija { ID = 81, KnjigaID = 18, Komentar = "Dobro" },
    new Recenzija { ID = 82, KnjigaID = 18, Komentar = "Uvredljivo" },
    new Recenzija { ID = 83, KnjigaID = 18, Komentar = "Okej knjiga" },
    new Recenzija { ID = 84, KnjigaID = 18, Komentar = "Preporučujem" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(18, "spam,uvredljivo");
        Assert.AreEqual(3, k.Recenzije.Count);
        Assert.AreEqual(3, _db.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 2");
    }
```

- Test 10: 5 ključnih riječi i jedna recenzija , podudaranje sa 4. tokenom (tokens.length=5, k.Recenzije.Count=1)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaCetvrtiPogadaJednuRecenziju()
    {
        var k = new Knjiga(20, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 90, KnjigaID = 20, Komentar = "Ovo je spam poruka" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(20, "aaa,bbb,ccc,spam,eee");
        Assert.AreEqual(0, k.Recenzije.Count);
        Assert.AreEqual(0, _db.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 1");
    }
```

- Test 11: 5 ključnih riječi i dvije recenzije , podudaranje sa 4. tokenom (tokens.length=5, k.Recenzije.Count=2)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaCetvrtiPogadaDvijeRecenzije()
    {
        var k = new Knjiga(21, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 100, KnjigaID = 21, Komentar = "Spam sadržaj" },
    new Recenzija { ID = 101, KnjigaID = 21, Komentar = "Odlična knjiga" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(21, "xxx,yyy,zzz,spam,www");
        Assert.AreEqual(1, k.Recenzije.Count);
        Assert.AreEqual(1, _db.Recenzije.Count);
        Assert.AreEqual("Odlična knjiga", k.Recenzije[0].Komentar);
        StringAssert.Contains(rezultat, "Uklonjeno 1");
    }
```

- Test 12: 5 ključnih riječi i 5 recenzija , podudaranje sa 4. tokenom (tokens.length=5, k.Recenzije.Count=5)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaCetvrtiPogadaPetRecenzija()
    {
        var k = new Knjiga(22, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 110, KnjigaID = 22, Komentar = "Spam 1" },
    new Recenzija { ID = 111, KnjigaID = 22, Komentar = "Dobra" },
    new Recenzija { ID = 112, KnjigaID = 22, Komentar = "Spam 2" },
    new Recenzija { ID = 113, KnjigaID = 22, Komentar = "Odlična" },
    new Recenzija { ID = 114, KnjigaID = 22, Komentar = "Super" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(22, "aaa,bbb,ccc,spam,eee");
        Assert.AreEqual(3, k.Recenzije.Count);
        Assert.AreEqual(3, _db.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 2");
    }
```

- Test 13: 5 ključnih riječi i 1 recenzija , podudaranje sa 5. tokenom (tokens.length=5, k.Recenzije.Count=1)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaPetiPogadaJednuRecenziju()
    {
        var k = new Knjiga(24, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 120, KnjigaID = 24, Komentar = "Ovo je spam" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(24, "aaa,bbb,ccc,ddd,spam");
        Assert.AreEqual(0, k.Recenzije.Count);
        Assert.AreEqual(0, _db.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 1");
    }
```

- Test 14: 5 ključnih riječi i 2 recenzije , podudaranje sa 5. tokenom (tokens.length=5, k.Recenzije.Count=2)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaPetiPogadaDvijeRecenzije()
    {
        var k = new Knjiga(25, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 130, KnjigaID = 25, Komentar = "Ima spam ovdje" },
    new Recenzija { ID = 131, KnjigaID = 25, Komentar = "Čista recenzija" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(25, "aaa,bbb,ccc,ddd,spam");
        Assert.AreEqual(1, k.Recenzije.Count);
        Assert.AreEqual(1, _db.Recenzije.Count);
        Assert.AreEqual("Čista recenzija", k.Recenzije[0].Komentar);
        StringAssert.Contains(rezultat, "Uklonjeno 1");
    }
```

- Test 15: 5 ključnih riječi i 5 recenzija , podudaranje sa 5. tokenom
  (tokens.length=5, k.Recenzije.Count=5)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaPetiPogadaPetRecenzija()
    {
        var k = new Knjiga(26, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 140, KnjigaID = 26, Komentar = "Spam ovdje" },
    new Recenzija { ID = 141, KnjigaID = 26, Komentar = "Još spam" },
    new Recenzija { ID = 142, KnjigaID = 26, Komentar = "Dobra knjiga" },
    new Recenzija { ID = 143, KnjigaID = 26, Komentar = "Odlično" },
    new Recenzija { ID = 144, KnjigaID = 26, Komentar = "Super" }
};
        _db.Recenzije.AddRange(k.Recenzije);
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(26, "aaa,bbb,ccc,ddd,spam");
        Assert.AreEqual(3, k.Recenzije.Count);
        Assert.AreEqual(3, _db.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 2");
    }
```

- Test 16: 5 ključnih riječi i jedna recenzija , nema podudaranja
  (tokens.length=5, k.Recenzije.Count=1)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaNemaPodudaranjaJednaRecenzija()
    {
        var k = new Knjiga(28, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 150, Komentar = "Normalna recenzija" }
};
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(28, "aaa,bbb,ccc,ddd,eee");
        Assert.AreEqual(1, k.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 0");
    }
```

- Test 17: 5 ključnih riječi i dvije recenzije , nema podudaranja
  (tokens.length=5, k.Recenzije.Count=2)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaNemaPodudaranjaDvijeRecenzije()
    {
        var k = new Knjiga(29, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 160, Komentar = "Normalna recenzija 1" },
    new Recenzija { ID = 161, Komentar = "Normalna recenzija 2" }
};
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(29, "aaa,bbb,ccc,ddd,eee");
        Assert.AreEqual(2, k.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 0");
    }
```

- Test 18: 5 ključnih riječi i 5 recenzija , nema podudaranja
  (tokens.length=5, k.Recenzije.Count=5)

```
    [TestMethod]
    0 references
    public void ObrisiNeprimjereneRecenzije_PetTokenaNemaPodudaranjaPetRecenzija()
    {
        var k = new Knjiga(30, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
        k.Recenzije = new List<Recenzija>
{
    new Recenzija { ID = 170, Komentar = "Recenzija 1" },
    new Recenzija { ID = 171, Komentar = "Recenzija 2" },
    new Recenzija { ID = 172, Komentar = "Recenzija 3" },
    new Recenzija { ID = 173, Komentar = "Recenzija 4" },
    new Recenzija { ID = 174, Komentar = "Recenzija 5" }
};
        _db.knjige.Add(k);
        string rezultat = _servis.ObrisiNeprimjereneRecenzije(30, "aaa,bbb,ccc,ddd,eee");
        Assert.AreEqual(5, k.Recenzije.Count);
        StringAssert.Contains(rezultat, "Uklonjeno 0");
    }
```

Petlja foreach iterira kroz listu recenzijeZaBrisanje i nezavisna je od prethodnih petlji:

Broj prolazaka	Opis	Pokriveno testom
0	Nema recenzija za brisanje	ObrisiNeprimjereneRecenzije_NemaPodudaranja
1	Jedna recenzija za brisanje	ObrisiNeprimjereneRecenzije_PetTokenaPetiPogadaPetRecenzija
2	Više recenzija za brisanje	ObrisiNeprimjereneRecenzije_UklanjaRecenzije

Obuhvat toka podataka

Testni slučajevi se dizajniraju tako da pokrivaju putanje od definicije do upotrebe (definition–use paths) varijable. Ova tehnika omogućava otkrivanje grešaka kao što su ne-inicijalizirane varijable ili pogrešno inicijalizirane.

1. Varijabla: k (tip: Knjiga)

- Definicija: var k = _db.GetKnjiga(idKnjige);
- Upotrebe:
  - if (k == null)
    - Pokriveno testom:
      ObrisiNeprimjereneRecenzije_KnjigaNePostoji_VracaGresku
      ObrisiNeprimjereneRecenzije_UklanjaRecenzije
  - if (k.Recenzije == null || k.Recenzije.Count == 0)
    - Pokriveno testom:
      ObrisiNeprimjereneRecenzije_PraznaListaRecenzija_VracaPoruku
      ObrisiNeprimjereneRecenzije_ListaRecenzijaNull_VracaPoruku

- for (int i = 0; i < k.Recenzije.Count; i++)
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- var recenzija = k.Recenzije[i];
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- k.Recenzije.Remove(recenzija);
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- return $"Uklonjeno {removed} neprimjerenih recenzija za knjigu '{k.NazivKnjige}'.";
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije

2. Varijabla: kljucneRijeci (tip: string - parametar metode)

- Definicija: Parametar metode
- Upotrebe:
  - if (IsNullOrWhite(kljucneRijeci))
    - Pokriveno testom: ObrisiNeprimjereneRecenzije_PrazneKljucneRijeci_VracaPoruku
  - var tokens = kljucneRijeci.Split(',', StringSplitOptions.RemoveEmptyEntries);
    - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije

3. Varijabla: tokens (tip: string[])

- Definicija: var tokens = kljucneRijeci.Split(',', StringSplitOptions.RemoveEmptyEntries);
- Upotrebe:
  - for (int t = 0; t < tokens.Length; t++)
    - Pokriveno testom: ObrisiNeprimjereneRecenzije_DvaTokenaDvijeRecenzijeSaPodudaranjem
  - var token = tokens[t].Trim();
    - Pokriveno testom: ObrisiNeprimjereneRecenzije_DvaTokenaDvijeRecenzijeSaPodudaranjem

4. Varijabla: recenzijeZaBrisanje (tip: List<Recenzija>)

- Definicija: var recenzijeZaBrisanje = new List<Recenzija>();
- Upotrebe:
  - recenzijeZaBrisanje.Add(recenzija);
    - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
  - int removed = recenzijeZaBrisanje.Count;
    - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
  - foreach (var recenzija in recenzijeZaBrisanje)
    - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije

5. Varijabla: i (tip: int - indeks vanjske petlje)

Definicija: for (int i = 0; i < k.Recenzije.Count; i++)
Upotrebe:

- i < k.Recenzije.Count
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- i++
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- var recenzija = k.Recenzije[i];
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije

6. Varijabla: recenzija (tip: Recenzija - iz vanjske petlje)

Definicija: var recenzija = k.Recenzije[i];
Upotrebe:

- var komentar = recenzija.Komentar ?? "";
  - Pokriveno testovima: ObrisiNeprimjereneRecenzije_KomentarNull ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- recenzijeZaBrisanje.Add(recenzija);
  - Pokriveno testovima: ObrisiNeprimjereneRecenzije_UklanjaRecenzije

7. Varijabla: komentar (tip: string)

Definicija: var komentar = recenzija.Komentar ?? "";
Upotrebe:

- if (CaseInsensitiveContains(komentar, token))
  - Pokriveno testovima: ObrisiNeprimjereneRecenzije_KomentarNull ObrisiNeprimjereneRecenzije_UklanjaRecenzije

8. Varijabla: t (tip: int - indeks unutrašnje petlje)

Definicija: for (int t = 0; t < tokens.Length; t++)
Upotrebe:

- t < tokens.Length
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- t++
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- var token = tokens[t].Trim();
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije

9. Varijabla: token (tip: string)

Definicija: var token = tokens[t].Trim();
Upotrebe:

- if (CaseInsensitiveContains(komentar, token))
  - Pokriveno testom: ObrisiNeprimjereneRecenzije_UklanjaRecenzije

10. Varijabla: removed (tip: int)

Definicija: int removed = recenzijeZaBrisanje.Count;

Upotrebe:

- if(removed > 0)
  - Pokriveno testovima:
    removed = 0: ObrisiNeprimjereneRecenzije_NemaPodudaranja
    removed > 0: ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- return $"Uklonjeno {removed} neprimjerenih recenzija za knjigu '{k.NazivKnjige}'.";
  - Pokriveno testom:
    ObrisiNeprimjereneRecenzije_UklanjaRecenzije

11. Varijabla: recenzija (tip: Recenzija - iz foreach petlje)

Definicija: foreach (var recenzija in recenzijeZaBrisanje)

Upotrebe:

- k.Recenzije.Remove(recenzija);
  - Pokriveno testovima:
    ObrisiNeprimjereneRecenzije_UklanjaRecenzije
- _db.Recenzije.Remove(recenzija);
  - Pokriveno testovima:
    ObrisiNeprimjereneRecenzije_UklanjaRecenzije

Sve relevantne DEF–USE putanje varijabli u metodi su pokrivene postojećim testnim slučajevima tj. nisu identifikovane putanje koje bi zahtijevale dodatne testove, te je obuhvat toka podataka postignut korištenjem već definisanog skupa testova.

Obuhvat puteva

Motivacija testiranja puteva je da se postigne kompletan obuhvat programskog koda testirajući sve njegove moguće puteve izvršavanja.
Mogući putevi u softverskim modulima se kreiraju u skladu sa uslovnim iskazima kao što su: IF–THEN–ELSE , DO WHILE, WHILE,....
Ovaj koncept nije praktičan u mnogim slučajevima jer je potrebno mnogo resursa i testnih slučajeva za njegovo izvršavanje. Obuhvat puteva razmatra se i u odnosu na linijski obuhvat.

McCabeova metrika je M = 11, pa je definisano tačno 11 nezavisnih putanja koje su logički moguće i koje zajedno pokrivaju svaku granu u grafu bar jednom:

Redni broj	Putanja	Opis scenarija	Pokriveno testom
1	1-2-3-24	Knjiga nije pronađena u bazi (k == null)	ObrisiNeprimjereneRecenzije_KnjigaNePostoji_VracaGresku
2	1-2-4-6-24	Lista recenzija je null	ObrisiNeprimjereneRecenzije_ListaRecenzijaNull_VracaPoruku

3	1-2-4-5-6-24	Lista recenzija je inicijalizovana, ali je prazna	ObrisiNeprimjereneRecenzije_PraznaListaRecenzija_VracaPoruku
4	1-2-4-5-7-8-24	Ključne riječi su prazne ili sadrže samo razmake	ObrisiNeprimjereneRecenzije_PrazneKljucneRijeci_VracaPoruku
5	1-2-4-5-7-9-10-11-12-14-18-10-19-21-23-24	Recenzija postoji, ali je komentar null. Unutrašnja petlja se preskače, nema brisanja	ObrisiNeprimjereneRecenzije_KomentarNull
6	1-2-4-5-7-9-10-11-12-13-14-18-10-19-21-23-24	Komentar nije null, ali je lista ključnih riječi prazna nakon split-a (tokens.Length == 0)	
7	1-2-4-5-7-9-10-11-12-13-14-15-16-14-18-10-19-21-23-24	Postoji recenzija i jedan token, ali se ne podudaraju. Nema brisanja	ObrisiNeprimjereneRecenzije_NemaPodudaranja
8	1-2-4-5-7-9-10-11-12-13-14-15-17-18-10-19-20-19-21-22-23-24	Pronađeno podudaranje u prvoj recenziji. Vrši se brisanje i poziva UpdateKnjige	ObrisiNeprimjereneRecenzije_JedanTokenJednaRecenzijaSaPodudaranjem
9	1-2-4-5-7-9-10-11-12-13-14-15-16-14-18-10-19-21-23-24	Druga iteracija vanjske petlje	ObrisiNeprimjereneRecenzije_JedanTokenDvijeRecenzije
10	1-2-4-5-7-9-10-11-12-13-14-15-16-14-15-17-18-10-19-20-19-21-22-23-24	Druga iteracija unutrašnje petlje (provjera drugog tokena ako prvi nije odgovarao)	ObrisiNeprimjereneRecenzije_PetTokenaCetvrtiPogadaJednuRecenziju
11	1-2-4-5-7-9-10-11-12-13-14-1	Višestruko brisanje u foreach petlji	ObrisiNeprimjereneRecenzije_UklanjaRecenzije

	5-17-18-10-11-12-13-14-15-17-18-10-19-20-19-20-19-21-22-23-24		

Put 6 (1-2-4-5-7-9-10-11-12-13-14-18-10-19-21-23-24) nije pokriven postojećim setom unit testova:

```
[TestMethod]
 0 references
public void ObrisiNeprimjereneRecenzije_AliPrazniTokeni_P6()
{
    var k = new Knjiga(40, "123", "Naziv", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"));
    k.Recenzije = new List<Recenzija> { new Recenzija { ID = 300, Komentar = "Onako , nije nešto." } };
    _db.knjige.Add(k);

    string rezultat = _servis.ObrisiNeprimjereneRecenzije(40, ",");

    StringAssert.Contains(rezultat, "Uklonjeno 0");
    Assert.AreEqual(1, k.Recenzije.Count);
}
```

Ostale identifikovane putanje ne zahtjevaju dodatne testove, te je obuhvat puteva postignut korištenjem već definisanog skupa testova i testa ObrisiNeprimjereneRecenzije_AliPrazniTokeni_P6

3. Izvršiti code tuning² za odabranu metodu primjenom minimalno 3 različite tehnike. Navesti koje su tehnike korištene i prikazati novi kod. Nakon svake urađene tehnike zabilježiti vrijeme izvršavanja i alociranu memoriju sa velikim brojem pokretanja. Zabilježiti i prodiskutovati i McCabe metriku i indeks održavanja te metode prije i poslije code tuning-a

Prije primjene tehnike **minimiziranje rada unutar petlje** :

```
var tokens = kljucneRijeci.Split(',', StringSplitOptions.RemoveEmptyEntries);

var recenzijeZaBrisanje = new List<Recenzija>();

for (int i = 0; i < k.Recenzije.Count; i++)
{
    var recenzija = k.Recenzije[i];
    var komentar = recenzija.Komentar ?? "";

    for (int t = 0; t < tokens.Length; t++)
    {
        var token = tokens[t].Trim();
        if (CaseInsensitiveContains(komentar, token))
        {
            recenzijeZaBrisanje.Add(recenzija);
            break;
        }
    }
}
```

```
Ukupno vrijeme za 10000 pokretanja: 2807,06 ms
Prosječno vrijeme po jednom brisanju: 0,2807 ms
Zauzeće memorije nakon testa: 61,41 KB
```

Poslije primjene tehnike **minimiziranje rada unutar petlje**:

```
var tokens = kljucneRijeci.Split(',', StringSplitOptions.RemoveEmptyEntries)
                          .Select(t => t.Trim())
                          .ToArray();

var recenzijeZaBrisanje = new List<Recenzija>();

for (int i = 0; i < k.Recenzije.Count; i++)
{
    var recenzija = k.Recenzije[i];
    var komentar = recenzija.Komentar ?? "";

    for (int t = 0; t < tokens.Length; t++){

        if (CaseInsensitiveContains(komentar, tokens[t])){
            recenzijeZaBrisanje.Add(recenzija);
            break;
        }
    }
}
```

```
ZAVRŠENO!
Ukupno vrijeme za 10000 pokretanja: 2585,84 ms
Prosječno vrijeme po jednom brisanju: 0,2586 ms
Zauzeće memorije nakon testa: 61,45 KB
```

Ova tehnika je primijenjena tako što je operacija Trim() izmještena iz unutrašnje petlje i izvršena samo jednom nad nizom tokena prije iteracija, umjesto da se ponavlja stotine puta za svaku recenziju.

Prije primjene tehnike **jaka redukcija:**

```
var komentar = recenzija.Komentar ?? "";

for (int t = 0; t < tokens.Length; t++){

    if (CaseInsensitiveContains(komentar, tokens[t])){
        recenzijeZaBrisanje.Add(recenzija);
        break;
    }
}
```

Poslije primjene tehnike **jaka redukcija**:

```
    var komentar = recenzija.Komentar ?? "";

    for (int t = 0; t < tokens.Length; t++){

        if (komentar.Contains(tokens[t], StringComparison.OrdinalIgnoreCase)){
            recenzijeZaBrisanje.Add(recenzija);
            break;
    }
    }
```

```
Ukupno vrijeme za 10000 pokretanja: 2512,35 ms
Prosječno vrijeme po jednom brisanju: 0,2512 ms
Zauzeće memorije nakon testa: 61,61 KB
```

Metoda CaseInsensitiveContains je zamijenjena sa efikasnijom i sistemski optimizovanom metodom .Contains (uz OrdinalIgnoreCase), čime je složena operacija zamijenjena jeftinijom ekvivalentnom operacijom.

Prije primjene tehnike **korištenje hešinga**:

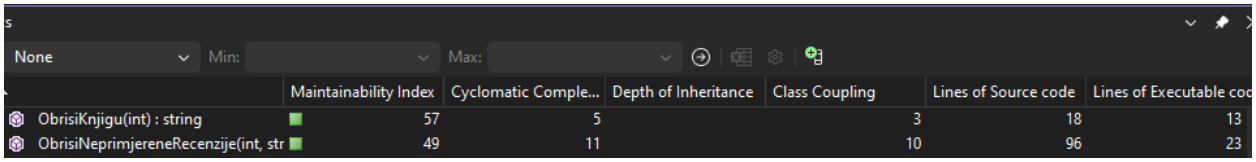
```
var recenzijeZaBrisanje = new List<Recenzija>();
```

Poslije primjene tehnike **korištenje hešinga**:

```
var recenzijeZaBrisanje = new HashSet<Recenzija>();
```

```
Ukupno vrijeme za 10000 pokretanja: 2477,04 ms
Prosječno vrijeme po jednom brisanju: 0,2477 ms
Zauzeće memorije nakon testa: 61,38 KB
```

Promjenom tipa kolekcije iz List u HashSet iskorištena je prednost heširanja za brži pristup podacima i automatsko uklanjanje duplikata bez potrebe za linearnim pretraživanjem niza.



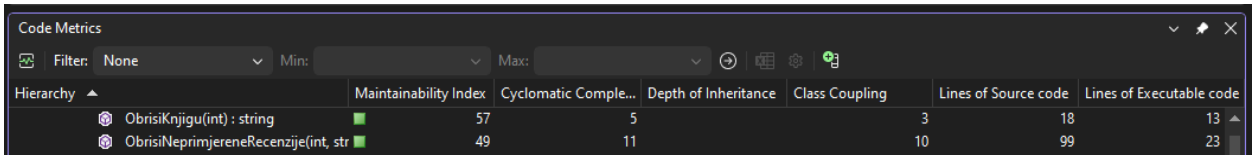
Optimizacijom metode poboljšala sam njeno izvršavanje i smanjila zauzeće memorije, čime su performanse značajno bolje, a indeks održivosti i ciklomatska kompleksnost su ostali isti.

4. Nakon code tuning-a, izvršiti refaktoring[3] tako da se postignu što bolje vrijednosti metrika. Prikazati kojoj stavki unutar checklist-e za refaktoring svaka promjena pripada, i kako ona utječe na metrike u sklopu Code Metrics.

Refaktoring na nivou podataka

- Rename a variable with a clearer or more informative name

Varijabla k je preimenovana u knjiga, a niz tokens u zabranjeneRijeci radi lakšeg razumijevanja. Opisna imena označavaju svrhu varijable bez potrebe za komentarima.



Refaktoring na nivou metode

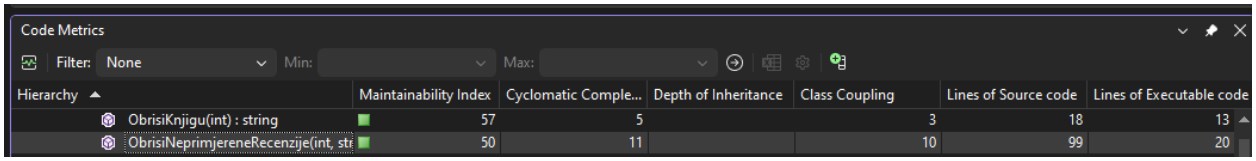
- Return as soon as you know the answer

Umjesto dubokog ugniježdenja, korišteni su Guard Clauses na početku metode. Ovo eliminiše potrebu za else blokovima i čini tok izvršavanja linearnim.

```
if (knjiga == null) return "GREŠKA: Knjiga nije pronađena.";

if (knjiga.Recenzije == null || knjiga.Recenzije.Count == 0)
    return "Nema recenzija za ovu knjigu.";

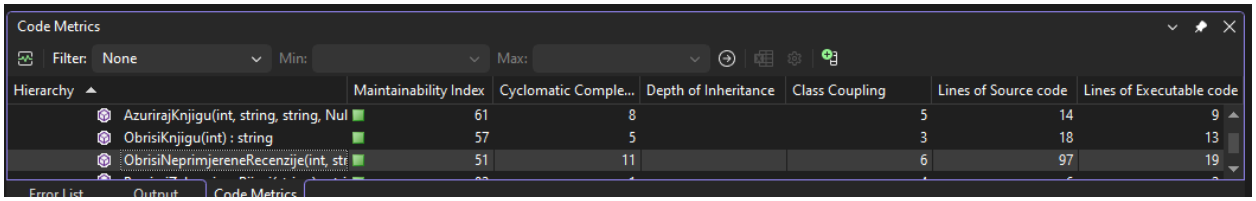
if (IsNullOrWhite(kljucneRijeci))
    return "Ključne riječi su prazne ili sadrže samo razmake. Nema brisanja.";
```



- Extract a routine

Logika parsiranja stringa u niz je izdvojena u zasebnu metodu jer predstavlja tehnički detalj koji ometa čitanje glavne poslovne logike.

```
private string[] ParsirajZabranjeneRijeci(string kljucneRijeci)
{
    return kljucneRijeci.Split(',', StringSplitOptions.RemoveEmptyEntries)
                        .Select(t => t.Trim())
                        .ToArray();
}
```

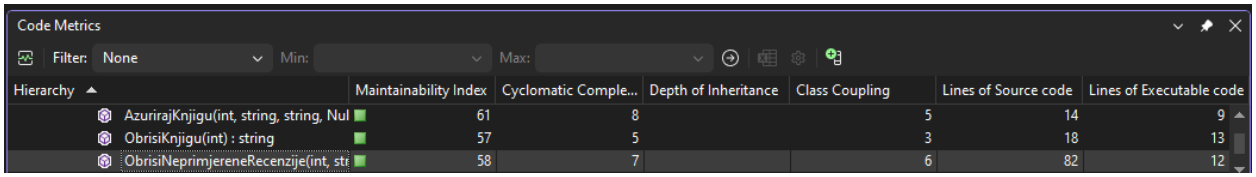


- Separate query operations from modification operations

Logika koja traži neprimjerene recenzije odvojena je od logike koja ih briše. Ovo omogućava lakše testiranje pretrage bez rizika od slučajnog brisanja podataka.

```
private HashSet<Recenzija> IdentifikujNeprimjereneRecenzije(IList<Recenzija> recenzije, string[] tokens)
{
    var zaBrisanje = new HashSet<Recenzija>();
    int count = recenzije.Count;

    for (int i = 0; i < count; i++)
    {
        var recenzija = recenzije[i];
        if (SadrziZabranjenuRijec(recenzija.Komentar, tokens))
        {
            zaBrisanje.Add(recenzija);
        }
    }
    return zaBrisanje;
}
```



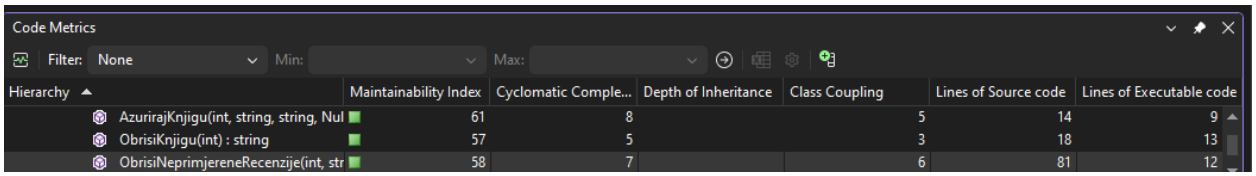
Refaktoring na nivou iskaza

- Decompose a boolean expression

Složeni uslov sa OrdinalIgnoreCase provjerom je izmješten u metodu SadrziZabranjenuRijec. Time se kompleksnost implementacije skriva, a kod postaje čitljiv kao rečenica.

```
private bool SadrziZabranjenuRijec(string komentar, string[] tokens)
{
    if (string.IsNullOrEmpty(komentar)) return false;

    for (int t = 0; t < tokens.Length; t++)
    {
        if (komentar.Contains(tokens[t], StringComparison.OrdinalIgnoreCase))
        {
            return true;
        }
    }
    return false;
}
```

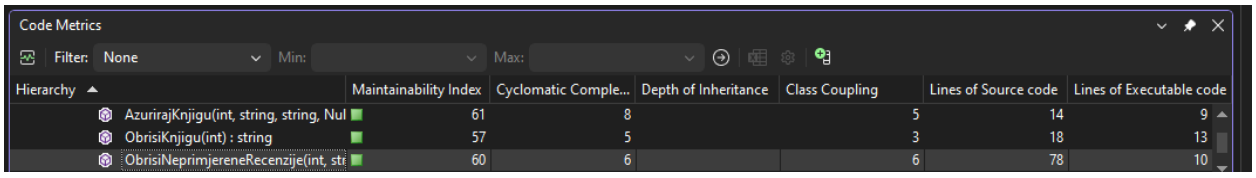


Refaktoring implementacije klase

- Changes within a class tend to be compartmentalized

Operacije nad bazom podataka su izolovane u posebnu metodu IzvrsiBrisanjeRecenzija. Ako se promijeni način rada sa bazom, mijenjamo samo ovu metodu, ne dirajući logiku pretrage.

```
private void IzvrsiBrisanjeRecenzija(Knjiga knjiga, HashSet<Recenzija> zaBrisanje)
{
    foreach (var r in zaBrisanje)
    {
        knjiga.Recenzije.Remove(r);
        _db.Recenzije.Remove(r);
    }
    _db.UpdateKnjige();
}
```



Kod nakon refaktoringa:

```
        public string ObrisiNeprimjereneRecenzije(int idKnjige, string kljucneRijeci)
        {
            var knjiga = _db.GetKnjiga(idKnjige);
            if (knjiga == null) return "GREŠKA: Knjiga nije pronađena.";

            if (knjiga.Recenzije == null || knjiga.Recenzije.Count == 0)
                return "Nema recenzija za ovu knjigu.";

            if (IsNullOrWhite(kljucneRijeci))
                return "Ključne riječi su prazne ili sadrže samo razmake. Nema
brisanja.";

            var zabranjeneRijeci = ParsirajZabranjeneRijeci(kljucneRijeci);

            var recenzijeZaBrisanje =
IdentifikujNeprimjereneRecenzije(knjiga.Recenzije, zabranjeneRijeci);

            int removed = recenzijeZaBrisanje.Count;

            if (removed > 0)
            {
                IzvrsiBrisanjeRecenzija(knjiga, recenzijeZaBrisanje);
            }

            return $"Uklonjeno {removed} neprimjerenih recenzija za knjigu
'{knjiga.NazivKnjige}'.";
        }
        private void IzvrsiBrisanjeRecenzija(Knjiga knjiga, HashSet<Recenzija>
zaBrisanje)
        {
            foreach (var r in zaBrisanje)
            {
                knjiga.Recenzije.Remove(r);
                _db.Recenzije.Remove(r);
            }
            _db.UpdateKnjige();
        }

        private HashSet<Recenzija> IdentifikujNeprimjereneRecenzije(IList<Recenzija>
recenzije, string[] zabranjeneRijeci)
        {
            var zaBrisanje = new HashSet<Recenzija>();

            for (int i = 0; i < recenzije.Count; i++)
            {
                var recenzija = recenzije[i];

                // Decompose boolean expression
                if (SadrziZabranjenuRijec(recenzija.Komentar, zabranjeneRijeci))
                {
                    zaBrisanje.Add(recenzija);
                }
            }
```

```
    return zaBrisanje;
}

private bool SadrziZabranjenuRijec(string komentar, string[] rijeci)
{
    if (string.IsNullOrEmpty(komentar)) return false;

    for (int t = 0; t < rijeci.Length; t++)
    {
        if (komentar.Contains(rijeci[t], StringComparison.OrdinalIgnoreCase))
        {
            return true;
        }
    }
    return false;
}

private string[] ParsirajZabranjeneRijeci(string kljucneRijeci)
{
    return kljucneRijeci.Split(',', StringSplitOptions.RemoveEmptyEntries)
                        .Select(t => t.Trim())
                        .ToArray();
}
```